

Git Branch

핵심 개념 및 실무 명령어 가이드

체크리스트

- 브랜치(Branch)의 물리적·개념적 특성 및 병렬 개발 흐름의 필요성 이해 여부
- 브랜치 생성, 조회, 삭제 CLI 명령어 능숙 사용 가능 여부
- `checkout` 대신 최신 권장 명령어인 `switch` 를 활용한 브랜치 전환 수행 가능 여부
- Fast-forward 병합과 3-way 병합의 발생 메커니즘 및 커밋 계통도 차이 구분 여부
- 병합 충돌(Conflict) 발생 원인 파악 및 충돌 마커 분석을 통한 수동 해결 가능 여부
- `git log --oneline --graph --all` 추적 가능 여부
- Squash, Rebase, Cherry-pick의 개념과 실무용도 구분 및 설명 가능 여부

브랜치란?

브랜치 (Branch)

동일한 프로젝트 코드베이스에서 서로 다른 작업(기능 추가, 버그 수정 등)을 독립적으로 병렬 진행하기 위해 분기하는 가상의 작업 흐름(가지).

🔗 브랜치의 물리적 실체 및 장점

가벼운 구조

대규모 파일 복제 방식이 아닌, 특정 커밋 해시
(40자리)를 가리키는 **4KB짜리 단순 텍스트 포인터**
파일에 불과함

성능 최적화

생성 및 전환 비용이 거의 발생하지 않아 민첩하고
신속한 가상 작업 공간 창출 지원

브랜치 다루기 (생성 및 전환)

1. 생성 및 목록 조회

- 브랜치 생성

```
git branch <새브랜치명>
```

현재 커밋 지점을 가리키는 포인터 생성

- 목록 조회

```
git branch
```

현재 사용 중인 활성 브랜치는 앞 ***** 기호와 함께 강조 표시됨

2. 브랜치 전환 (switch 중심)

배경: 기존 `checkout` 이 브랜치 전환/파일 복원 등
너무 많은 책임을 안고 있어 분리됨

- 브랜치 전환 (실무 권장)

```
git switch <이동할브랜치명>
```

- 생성 및 전환 동시 수행

```
git switch -c <새브랜치명>
```

- 파일 변경 사항 복원

```
git restore <파일명>
```

브랜치 다루기 (삭제)

3. 브랜치 삭제

일반 삭제

```
git branch -d <삭제할브랜치명>
```

대상 브랜치에 병합(Merge)이 완료된 안전한 상태인 경우만 허용

강제 삭제

```
git branch -D <삭제할브랜치명>
```

병합 여부와 관계없이 작업 내역을 강제로 즉시 삭제

브랜치 병합 (Merge) (1/2)

작업 완료 후, 분기된 브랜치의 작업 내역을 기준 브랜치(예: `main`, `develop`)에 통합하는 프로세스.

1. Fast-Forward Merge (빨리감기 병합)

- **조건:** 분기 브랜치 생성 이후, 기준 브랜치(Base)에 아무런 추가 커밋이 발생하지 않은 상태
- **방식:** 기준 브랜치의 포인터를 분기 브랜치의 최신 커밋 위치로 단순히 앞으로 신속히 이동시킴
- **특징:** 별도의 병합 커밋(Merge Commit)이 생성되지 않고 외줄의 깨끗한 히스토리가 유지됨

브랜치 병합 (Merge) (2/2)

2. 3-way Merge (3방향 병합)

- **조건:** 분기 이후 기준 브랜치와 분기 브랜치 양측 모두에 독립적인 추가 커밋이 각각 발생한 상태
- **방식:** 기준 브랜치 최신 커밋, 분기 브랜치 최신 커밋, 그리고 공통 조상(Common Ancestor) 커밋 등 총 3개 지점을 비교 융합
- **특징:** 병합 완료 시 자동으로 새로운 **병합 커밋(Merge Commit)**이 계통도에 생성됨

병합 충돌 (Conflict) (1/2)

- **원인:** 3-way Merge 시 동일 파일의 동일 라인(Line)을 양쪽 브랜치에서 서로 다르게 수정하고 커밋한 경우
- **동작:** Git이 임의로 덮어쓰지 않고 충돌 마커를 표시한 채 병합을 일시 중단하며 사용자 수동 해결 대기

충돌 마커 구성

```
<<<<<<< HEAD
현재 작업 브랜치(예: main)의 변경 내용
=====
가져와 병합하려는 대상 브랜치(예: feature)의 변경 내용
>>>>>>> feature
```


병합 충돌 (Conflict) (2/2)

해결 3단계 프로토콜

1. 충돌 파일 수동 수정: 마커 기호 및 불필요한 코드를 깔끔히 지우고 최적 코드로 편집
2. 수정 파일 스테이징: `git add <파일명>`
3. 병합 완료 커밋: `git commit` (자동 완성된 메시지 그대로 반영)

브랜치 정리 (고급 개념)

히스토리를 아름답고 읽기 쉽게 정돈하기 위한 실무 3대 정리 기법

1. Squash

스쿼시

개념: 작업 브랜치의 수많은 잔가지 커밋들을 단 하나의 단일 커밋으로 압축하여 병합

효과: 메인 히스토리가 파편화되지 않고 깔끔하게 통합 관리됨

2. Rebase

리베이스

개념: 내 브랜치의 출발 지점(Base)을 대상 브랜치의 최신 커밋 지점으로 새롭게 갱신

효과: 복잡한 그물망식 병합선을 없애고 하나의 선형(직선) 히스토리로 정돈함

3. Cherry-pick

체리피킹

개념: 타 브랜치에 기록된 수많은 커밋 중 원하는 특정 커밋 단 하나만 선택적으로 복제하여 이식

효과: 긴급 버그 패치나 특정 기능 코드의 빠른 부분 반영 시 매우 유용



인터랙티브 실습 시뮬레이터

Learn Git Branching

- **특징:** 웹 브라우저 상의 터미널 명령어 입력에 맞추어 브랜치 포인터와 노드가 실시간 애니메이션으로 반응하는 비주얼 교육 게임
- **학습법:** 준비된 핵심 미션 시나리오를 단계별로 완수하며 브랜치 생성·이동·병합 및 충돌 해결 흐름을 직관적으로 학습 가능